

Lab 5: Quantum Classifier

QML

1 introduction

this is it — the capstone lab. we r building actual quantum machine learning classifiers and comparing them head-to-head with the classical baselines from lab 4. u'll implement two approaches:

1. variational quantum classifier (VQC) — a parameterized quantum circuit trained with gradient descent, similar to a neural network
2. quantum support vector classifier (QSVC) — uses a quantum kernel computed on a quantum computer, plugged into a classical SVM

both approaches use quantum feature maps to encode classical data into quantum states. the question is: does the quantum feature map provide any advantage over classical feature maps?

by the end of this lab u should be able to:

- build quantum feature maps (data encoding circuits)
- implement a VQC from scratch using qiskit
- compute quantum kernel matrices
- train a QSVC using the precomputed kernel workflow from lab 4
- rigorously compare quantum vs classical classifiers

Key Point. we r using the same iris dataset (binary, 2 features) as lab 4. same preprocessing, same train/test split. this way the comparison is fair.

2 quantum feature maps

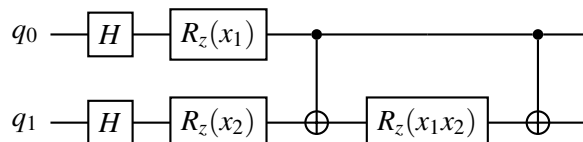
2.1 encoding classical data

the first step in any quantum ML pipeline: encode classical data $x \in \mathbb{R}^d$ into a quantum state $|\phi(x)\rangle$. this is done by a parameterized circuit where the parameters r set by the data.

Definition 2.1. a quantum feature map is a unitary $U(x)$ that maps classical data x to a quantum state:

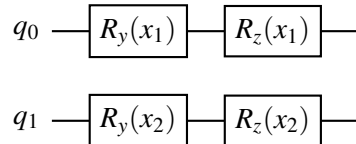
$$|\phi(x)\rangle = U(x) |0\rangle^{\otimes n}$$

the most common feature map is the ZZ feature map. for 2 qubits and 2 features (x_1, x_2) :

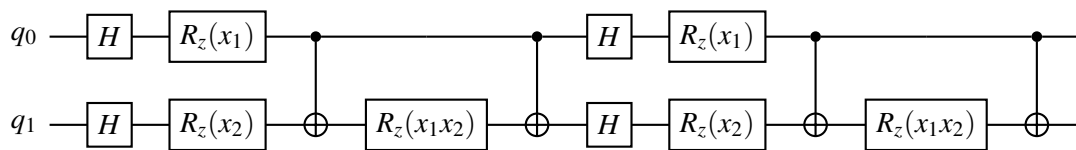


the H gates create superposition. the $R_z(x_i)$ gates encode individual features. the CNOT- R_z -CNOT block encodes the interaction $x_1 \cdot x_2$. this is important bcs it means the feature map captures correlations between features, not just individual features.

another option is the angle encoding feature map, which is simpler:



and u can repeat the feature map multiple times (called “data re-uploading”) to make it more expressive:



Exercise 1. building feature maps.

1. implement the ZZ feature map for 2 qubits manually:

- create a `QuantumCircuit(2)`
- apply H to both qubits
- apply $R_z(x_1)$ to qubit 0 and $R_z(x_2)$ to qubit 1
- apply $CNOT(0,1)$, then $R_z(x_1 \cdot x_2)$ on qubit 1, then $CNOT(0,1)$

2. also implement the angle encoding feature map

3. test both with a sample data point, e.g., $x = (0.5, -0.3)$

4. use the statevector simulator to get $|\phi(x)\rangle$ for each feature map

5. try a few different data points and see how the output states differ

u can also use qiskit’s built-in feature maps: `ZZFeatureMap` and `ZFeatureMap` from `qiskit.circuit.library`. compare ur manual implementation with the built-in one.

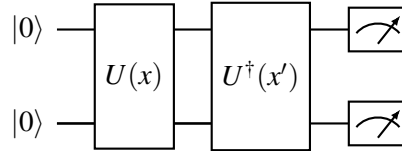
3 quantum kernel method (QSVC)

3.1 the quantum kernel

the quantum kernel between two data points x and x' is:

$$k(x, x') = |\langle \phi(x) | \phi(x') \rangle|^2$$

this measures how similar the quantum states r for two different data points. to compute this on a quantum computer, we use the following circuit:



u apply $U(x)$ to encode the first data point, then $U^\dagger(x')$ to “undo” the second data point. then u measure: the probability of getting all zeros is exactly $|\langle \phi(x) | \phi(x') \rangle|^2$.

Intuition. if $x = x'$, the circuit does $U(x)U^\dagger(x) = I$, so u always measure $|00\dots 0\rangle$ and the kernel is 1. if x and x' r very different, the states will be orthogonal and the kernel will be close to 0. so the kernel measures similarity in quantum state space.

Exercise 1. computing the quantum kernel.

1. write a function `quantum_kernel(x, x_prime, feature_map)` that:
 - applies the feature map $U(x)$
 - applies the inverse feature map $U^\dagger(x')$
 - measures all qubits
 - returns the probability of the all-zeros outcome
2. test it: compute $k(x,x)$ for some point x . u should get ≈ 1 .
3. compute $k(x,x')$ for two very different points. should be much less than 1.
4. now write a function `quantum_kernel_matrix(X1, X2, feature_map)` that computes the full kernel matrix. this requires $|X1| \times |X2|$ circuit evaluations.
5. compute the kernel matrix for the training data. visualize it as a heatmap. compare with the RBF kernel heatmap from lab 4.

Exercise 2. training QSVC.

1. compute the quantum kernel matrix for training data: $K_{\text{train}}(i, j) = k(x_i, x_j)$
2. compute the quantum kernel matrix for test data: $K_{\text{test}}(i, j) = k(x_i^{\text{test}}, x_j^{\text{train}})$
3. use `SVC(kernel='precomputed')` from sklearn (exactly like exercise in lab 4)
4. train: `.fit(K_train, y_train)`
5. predict: `.predict(K_test)`
6. report test accuracy
7. compare with ur classical SVM baseline from lab 4

use the ZZ feature map with 1 repetition first. then try 2 repetitions. does more repetitions help?

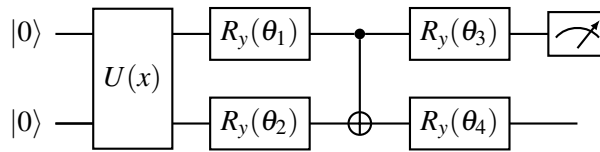
4 variational quantum classifier (VQC)

4.1 the architecture

a VQC has three parts:

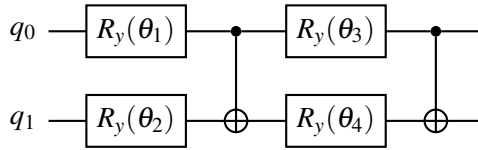
1. feature map $U(x)$ — encodes the input data (same as before)
2. variational ansatz $V(\theta)$ — parameterized circuit with trainable parameters
3. measurement — measure one qubit and interpret as the class label

the full circuit for a 2-qubit VQC:



the feature map encodes the data, the ansatz has trainable parameters θ , and we measure qubit 0. if $P(|0\rangle) > 0.5$, predict class 0; otherwise predict class 1.

the ansatz can have multiple layers. a single layer typically has R_y rotations on each qubit followed by CNOT entangling gates. here's a 2-layer ansatz:



4.2 training

training a VQC is an optimization problem:

$$\min_{\theta} \mathcal{L}(\theta) = \min_{\theta} \frac{1}{m} \sum_{i=1}^m \ell(f_{\theta}(x_i), y_i)$$

where $f_{\theta}(x)$ is the VQC's prediction and ℓ is a loss function (e.g., cross-entropy or MSE).

the challenge: computing gradients of quantum circuits. we use the parameter shift rule:

$$\frac{\partial}{\partial \theta_k} f(\theta) = \frac{1}{2} \left[f\left(\theta_k + \frac{\pi}{2}\right) - f\left(\theta_k - \frac{\pi}{2}\right) \right]$$

this means: to compute the gradient w.r.t. one parameter, u run the circuit twice (with the parameter shifted by $\pm\pi/2$). for p parameters, u need $2p$ circuit evaluations per gradient step.

Exercise 1. building the VQC.

1. build the VQC circuit manually:

- create a parameterized circuit with qiskit `Parameter` objects

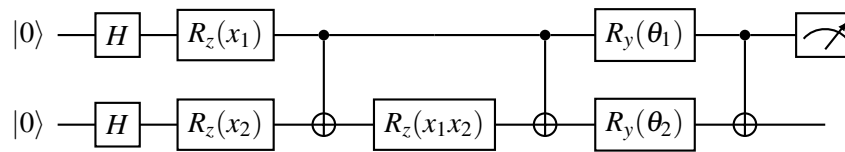
- first apply the ZZ feature map (with data parameters)
- then apply a variational ansatz: 2 layers of R_y rotations + CNOT entangling
- add measurement on qubit 0

2. write a function `predict(x, theta)` that:

- binds the data parameters to x and the variational parameters to θ
- runs the circuit on the qasm simulator
- returns $P(|0\rangle)$ as the prediction

3. test with random θ values. the predictions should be between 0 and 1.

the circuit should look like this (feature map + 1-layer ansatz + measurement):



Exercise 2. training the VQC.

1. implement the loss function: binary cross-entropy

$$\mathcal{L} = -\frac{1}{m} \sum_i [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

where $p_i = \text{predict}(x_i, \theta)$

2. implement the parameter shift rule to compute gradients:

- for each parameter θ_k , compute the loss at $\theta_k + \pi/2$ and $\theta_k - \pi/2$
- the gradient is the difference divided by 2

3. implement gradient descent:

- initialize θ randomly (small values, e.g., uniform in $[0, \pi]$)
- learning rate: start with 0.1
- run for 50–100 iterations
- record the loss at each step

4. plot the training loss curve. does it converge?

5. compute test accuracy. how does it compare with the classical SVM?

tips:

- training is slow bcs each gradient step requires $2p \times m$ circuit evaluations (p parameters, m training samples). use the statevector simulator to speed things up.
- if training is too slow, subsample the training data (use 30–40 points).
- try different random initializations — variational circuits can get stuck in local minima.

Exercise 3. VQC hyperparameters. explore how these choices affect performance:

1. number of ansatz layers (1, 2, 3). more layers = more parameters = more expressive, but also harder to train and more noise.
2. feature map type: ZZ feature map vs angle encoding. which works better?
3. number of shots (100, 1000, 10000). more shots = lower variance but slower.
4. learning rate (0.01, 0.1, 0.5). too high and it oscillates, too low and it's slow.
5. optimizer: try both vanilla gradient descent and COBYLA (`scipy.optimize.minimize` with `method='COBYLA'`). COBYLA is gradient-free so it might work better if gradients are noisy.

5 the big comparison

Exercise 1. quantum vs classical showdown. this is the main event. compile your results into a comparison table:

classifier	test accuracy	training time
classical linear SVM		
classical RBF SVM		
QSVC (ZZ feature map, 1 rep)		
QSVC (ZZ feature map, 2 reps)		
VQC (1 layer)		
VQC (2 layers)		

answer these questions:

1. does the quantum classifier beat the classical one on this dataset?
2. if not, does that mean quantum ML is useless? what factors could change the outcome?
3. which quantum approach (VQC or QSVC) works better? why might that be?
4. what's the computational cost comparison? how many circuit evaluations did QSVC require vs VQC?
5. if you added noise (from lab 3), how would it affect each quantum method?

6 bonus: noisy quantum classifier

Exercise 1. running with noise.

1. repeat the QSVC experiment but with a noisy simulator (depolarizing noise, $p = 0.01$ single-qubit, $p = 0.05$ CNOT)
2. how much does accuracy drop?

3. apply measurement error mitigation from lab 3 to the kernel computation
4. does mitigation help recover the accuracy?
5. now try ZNE on the kernel circuits. is it worth the extra computational cost?

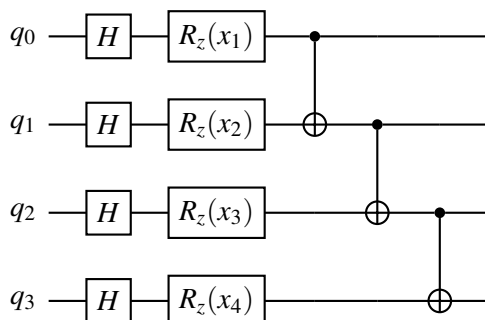
this exercise shows u the real challenge of near-term quantum ML: the noise can easily wipe out any advantage the quantum feature map might have.

7 bonus: scaling up

Exercise 1. more features.

1. repeat the QSVC experiment using all 4 iris features instead of 2 (u'll need 4 qubits)
2. does using more features help the quantum classifier?
3. compare the 4-feature quantum kernel with the 4-feature classical RBF kernel
4. the 4-qubit feature map lives in a $2^4 = 16$ dimensional Hilbert space. the RBF kernel implicitly uses infinite dimensions. discuss when the quantum approach might still win despite having “fewer” dimensions.

4-qubit ZZ feature map (first layer only, without interaction terms for clarity):



8 final reflection

Exercise 1. course synthesis. write a 1-page reflection addressing:

1. the pipeline: trace the full QML pipeline from data preprocessing → feature map → quantum circuit → measurement → classical post-processing. where does quantum help? where is it classical?
2. the bottleneck: wt's the main practical limitation of quantum ML right now? (noise? number of qubits? training time? all of the above?)
3. the promise: under wt conditions might quantum ML provide a genuine advantage? think about data structure, feature space dimensionality, and kernel expressiveness.
4. ur experience: wt was the most surprising thing u learned in this course? wt would u want to explore further?

9 submission

submit a jupyter notebook with:

- all exercises completed with code and output
- the full comparison table filled in
- kernel matrix heatmaps (quantum vs classical)
- training loss curves for the VQC
- decision boundary plots where possible
- ur final reflection

this notebook should tell a complete story: classical baseline $\rightarrow$ quantum kernel $\rightarrow$ quantum variational $\rightarrow$ comparison $\rightarrow$ reflection.